

USE CASE 2 — CRIME TREND AND PATTERN ANALYSIS

Name: Bhimireddy Bharath kumar Reddy

Email: bhimireddy.bharath99@gmail.com

Objective: Analyze the cleaned Chicago crime dataset to identify crime trends, crime categories, arrest patterns, monthly and weekly crime patterns, and high-crime community areas.

Tasks :

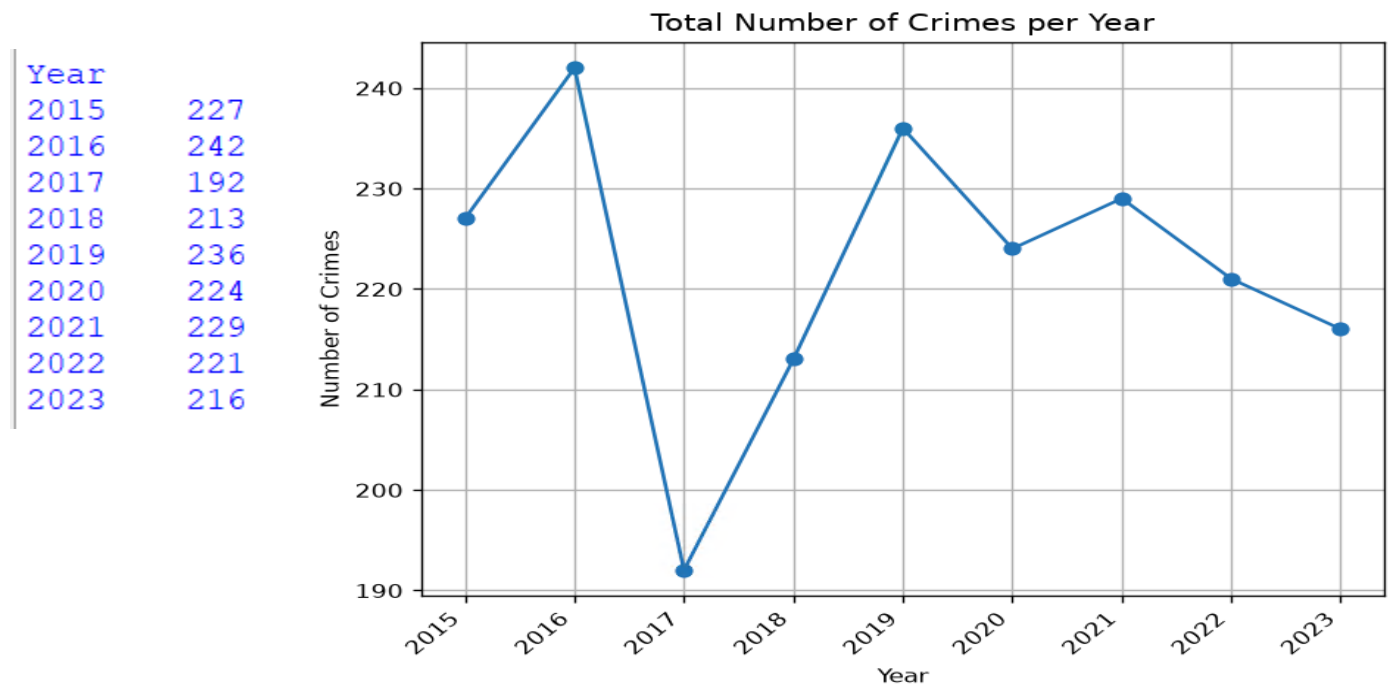
1. Crime Trend Over Years

1.1 Plot Total Number of Crimes Per Year

Code:

```
def analyze_yearly_crimes(data):  
    print_title("1. CRIME TREND OVER YEARS")  
    print_task("1.1", "PLOT TOTAL NUMBER OF CRIMES PER YEAR")  
    yearly_count = (data.groupby("Year").size().sort_index())  
    print(yearly_count.to_string())  
    display_graph(yearly_count.index, yearly_count.values,  
                  "Total Number of Crimes per Year", "Year",  
                  "Number of Crimes")
```

Output:



Insight:

- The graph shows how the total number of reported crimes changes from year to year.
- It helps identify whether the overall crime trend is increasing, decreasing, or relatively stable.
- The yearly trend is determined using the number of crime records for each year.

1.2 Visual Interpretation — Is Crime Rate Rising or Decreasing?

Code:

```
print_task("1.2",
    "VISUAL INTERPRETATION : IS CRIME RATE RISING OR DECREASING ?" )
# Calculate the slope of the yearly crime trend
years = yearly_count.index.to_numpy()
crime_values = yearly_count.values
slope = np.polyfit(years, crime_values, 1)[0]
if slope > 0:
    print("Overall crime trend is rising.")
elif slope < 0:
    print("Overall crime trend is decreasing.")
else:
    print("Overall crime trend is stable.")
```

Output:

```
Overall crime trend is decreasing.
```

Insight:

- The overall trend is determined by calculating the slope of the yearly crime counts.
- A positive slope indicates an increasing crime trend.
- A negative slope indicates a decreasing crime trend.
- A zero or near-zero slope indicates a relatively stable trend.

2. Crime Distribution by Category

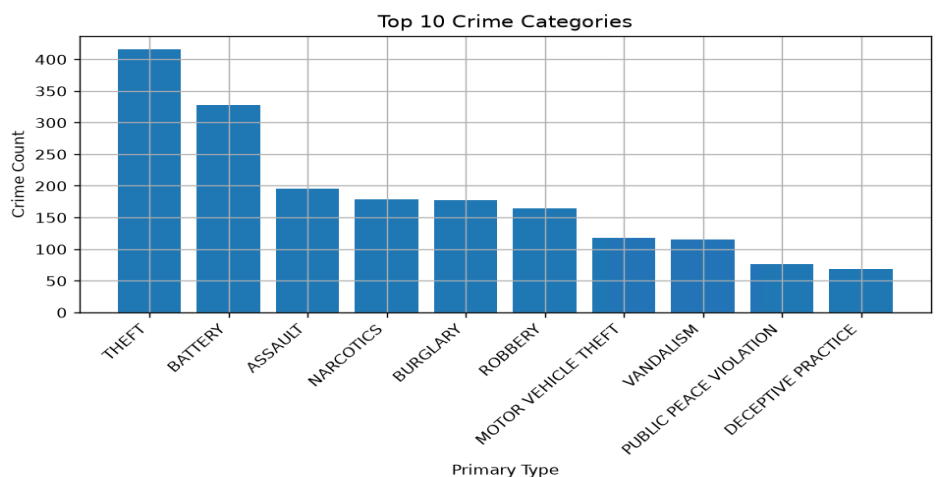
2.1 Top 10 Crime Categories

Code :

```
def study_crime_categories(data):
    print_title("2. CRIME DISTRIBUTION BY CATEGORY")
    crime_frequency = (data["primary_type"].value_counts())
    highest_ten = crime_frequency.iloc[:10]
    print_task( "2.1","BAR CHART TOP 10 CRIME CATEGORIES (PRIMARY TYPE)")
    print(highest_ten.to_string())
    display_graph(highest_ten.index,highest_ten.values,
        "Top 10 Crime Categories","Primary Type",
        "Crime Count",graph_type="bar")
```

Output :

primary_type	
THEFT	416
BATTERY	328
ASSAULT	195
NARCOTICS	178
BURGLARY	177
ROBBERY	165
MOTOR VEHICLE THEFT	118
VANDALISM	115
PUBLIC PEACE VIOLATION	76
DECEPTIVE PRACTICE	68



Insight:

- The bar chart displays the 10 most frequent crime categories in the dataset.
- It helps identify which types of crimes occur most frequently.
- The categories are ranked based on their total number of records.

2.2 Crime Count and Percentage for Each Category

Code :

```
print_task("2.2", "CALCULATE COUNTS AND PERCENTAGE FOR EACH")
percentage_values = (crime_frequency.div(len(data))
                    .mul(100).round(2))
category_summary = pd.concat(
    [
        crime_frequency.rename("Count"), percentage_values.rename("Percentage")
    ], axis=1)
print(category_summary.to_string())
```

Output:

primary_type	Count	Percentage
THEFT	416	20.80
BATTERY	328	16.40
ASSAULT	195	9.75
NARCOTICS	178	8.90
BURGLARY	177	8.85
ROBBERY	165	8.25
MOTOR VEHICLE THEFT	118	5.90
VANDALISM	115	5.75
PUBLIC PEACE VIOLATION	76	3.80
DECEPTIVE PRACTICE	68	3.40
WEAPONS VIOLATION	50	2.50
CRIMINAL TRESPASS	50	2.50
SEX OFFENSE	21	1.05
ARSON	20	1.00
CRIM SEXUAL ASSAULT	13	0.65
HOMICIDE	10	0.50

Insight:

- The analysis calculates both the total number of records and percentage contribution for each crime category.
- This provides a better understanding of the distribution of different crime types.
- Higher percentages indicate crime categories that form a larger portion of the dataset.

3. Arrests and Crime Outcomes

3.1 What Percentage of Crime Results in Arrest?

Code :

```
def examine_arrests(data):
    print_title("3. ARRESTS AND CRIME OUTCOMES")
    arrest_values = data["arrest"]
    arrest_percentage = (arrest_values.sum() / len(arrest_values) * 100)
    print_task("3.1", "WHAT PERCENTAGE OF CRIME RESULTS IN ARREST?")
    print(f"Arrest percentage: {arrest_percentage:.2f}%")
```

Output:

```
Arrest percentage: 29.80%
```

Insight:

- The arrest rate represents the percentage of crime records where an arrest was recorded.
- It provides an overview of the proportion of reported crimes associated with arrests.
- This helps understand the relationship between reported crimes and arrest outcomes.

3.2 Display Arrest Percentage

Code :

```
print_task("3.2", "DISPLAY ARREST PERCENTAGE")
print(f"arrest_rate = {arrest_percentage:.2f}%")
```

Output:

```
arrest_rate = 29.80%
```

Insight:

- The calculated percentage gives a simple numerical measure of the arrest outcome.
- It can be used as a baseline for comparing arrest patterns across different years.

4. Heatmap of Crime by Month and Day of Week

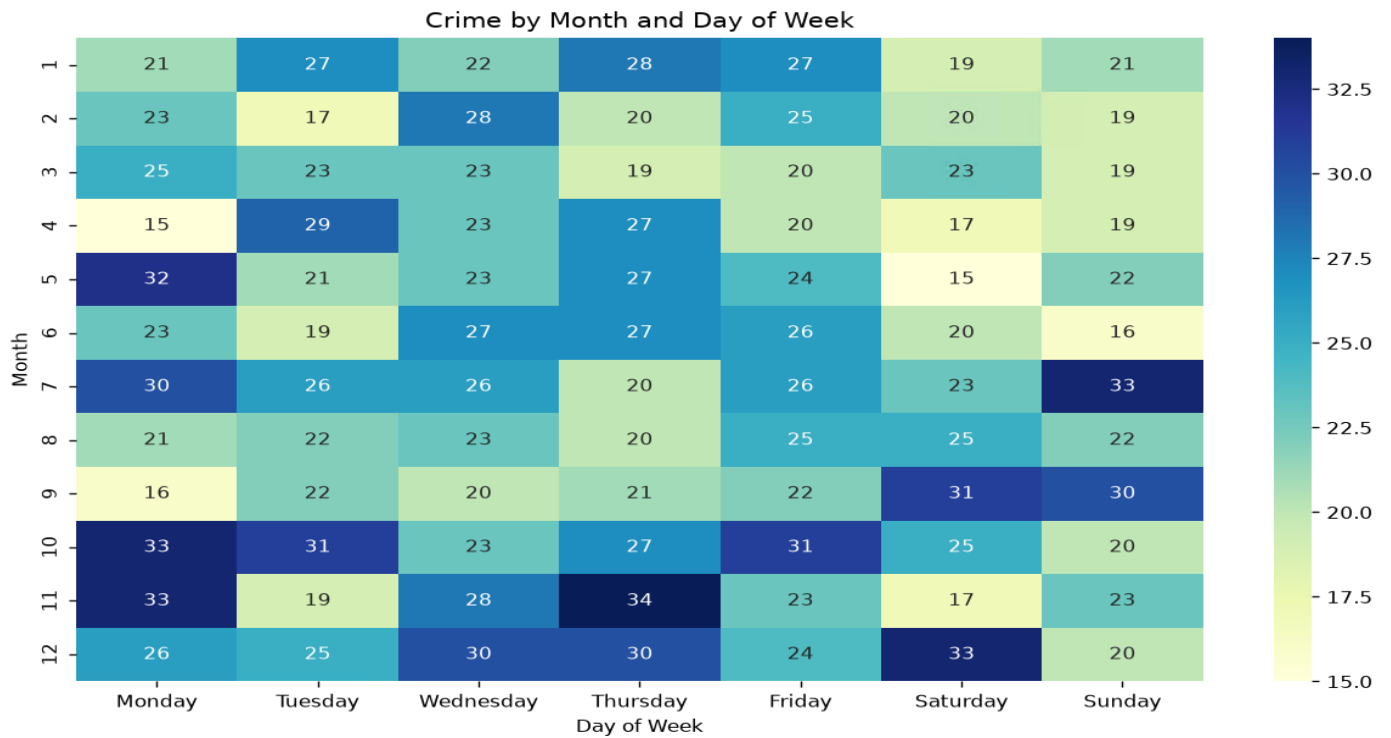
4.1 Crime Frequency by Month and Day of Week

Code :

```
def create_crime_heatmap(data):
    print_title("4. HEATMAP OF CRIME BY MONTH AND DAY OF WEEK")
    print_task("4.1", "USE SEABORN HEATMAP OF CRIME FREQUENCY "
              "PIVOTED BY MONTH VS DAYOFWEEK")
    # Support either spelling used by the cleaned dataset
    if "DayOfWeek" in data.columns:
        weekday_column = "DayOfWeek"
    else:
        weekday_column = "Dayofweek"
    weekday_order = [ "Monday", "Tuesday", "Wednesday",
                      "Thursday", "Friday", "Saturday", "Sunday"]
    month_day_table = pd.crosstab(data["Month"], data[weekday_column])
    month_day_table = month_day_table.reindex(columns=weekday_order,
                                              fill_value=0)
    print(month_day_table.to_string())
    plt.figure(figsize=(10, 6))
    sns.heatmap(month_day_table, annot=True, fmt="d", cmap="YlGnBu")
    plt.title("Crime by Month and Day of Week")
    plt.xlabel("Day of Week")
    plt.ylabel("Month")
    plt.tight_layout()
    plt.show()
```

Output:

DayOfWeek	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
Month							
1	21	27	22	28	27	19	21
2	23	17	28	20	25	20	19
3	25	23	23	19	20	23	19
4	15	29	23	27	20	17	19
5	32	21	23	27	24	15	22
6	23	19	27	27	26	20	16
7	30	26	26	20	26	23	33
8	21	22	23	20	25	25	22
9	16	22	20	21	22	31	30
10	33	31	23	27	31	25	20
11	33	19	28	34	23	17	23
12	26	25	30	30	24	33	20



Insight:

- The heatmap shows crime frequency across different months and days of the week.
- Darker or higher-valued cells represent areas with more crime records.
- This helps identify periods when crime occurs more frequently.
- The analysis can reveal recurring monthly or weekly crime patterns.

5. Community Area Analysis

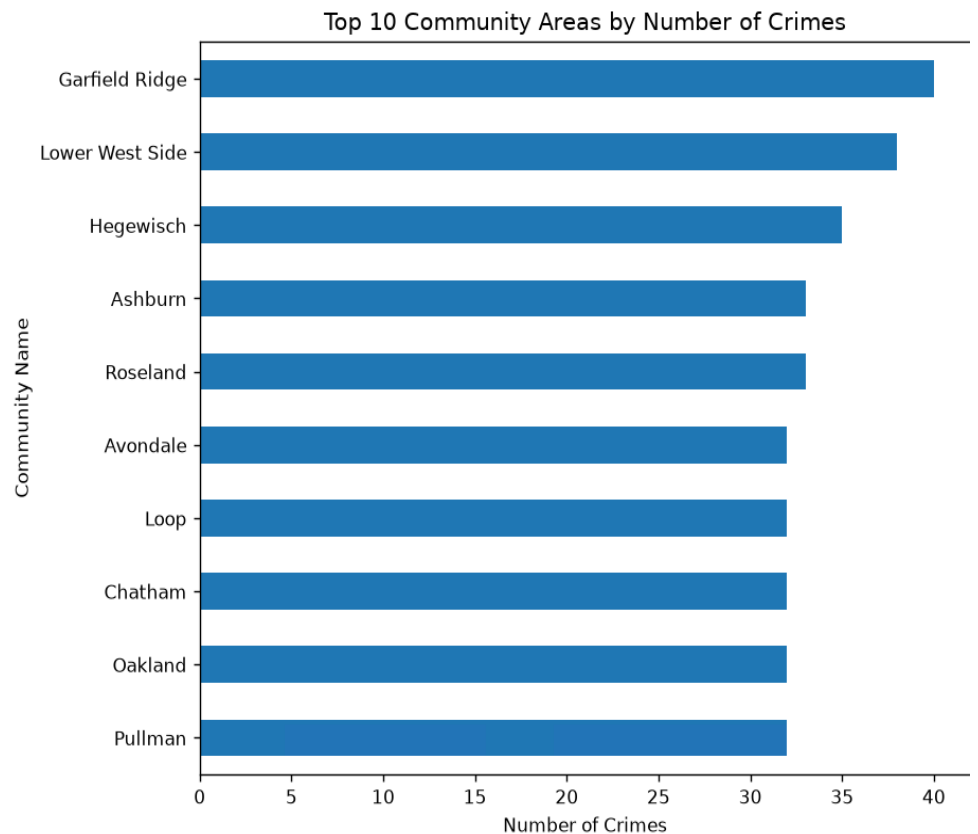
5.1 Top 10 Community Areas by Number of Crimes

Code :

```
def community_analysis(df):
    print("\n" + "=" * 70)
    print("COMMUNITY AREA ANALYSIS")
    print("=" * 70)
    # Find community CSV without using an absolute path
    community_file = find_dataset_file("chicago_city_community.csv")
    community_df = pd.read_csv(community_file)
    print("\nAvailable columns:")
    print(community_df.columns.tolist())
    # Select required columns
    community_df = community_df[["community_code", "community_name"]].copy()
    # Convert codes to numeric
    df["community_code"] = pd.to_numeric(df["community_code"], errors="coerce")
    community_df["community_code"] = pd.to_numeric(community_df["community_code"],
                                                    errors="coerce")
    # Add community names to crime data
    merged_df = df.merge(community_df, on="community_code", how="left")
    # Top 10 communities
    top_10 = (merged_df.groupby("community_name").size()
              .sort_values(ascending=False).head(10))
    print("\nTop 10 Community Areas:")
    print(top_10)
    # Graph
    plt.figure(figsize=(12, 6))
    top_10.sort_values().plot(kind="barh")
    plt.title("Top 10 Community Areas by Number of Crimes")
    plt.xlabel("Number of Crimes")
    plt.ylabel("Community Name")
    plt.tight_layout()
    plt.show()
    return merged_df
```

Output:

```
Top 10 Community Areas:
community_name
Garfield Ridge      40
Lower West Side     38
Hegewisch           35
Roseland            33
Ashburn             33
Chatham            32
Pullman            32
Oakland            32
Loop               32
Avondale           32
dtype: int64
```



Insight:

- Crime records are matched with community names using the community code.
- The analysis identifies the top 10 community areas with the highest number of crime records.
- This helps identify communities with comparatively higher reported crime activity.
- The result can be useful for understanding the geographical distribution of crime.

Questions :

Q1. Which Crime Category Is Most Frequent?

Code :

```
def solve_questions(data):
    print_title("QUESTIONS")
    # -----
    # Q1
    # -----
    print_task("Q1.", "WHICH CRIME CATEGORY IS MOST FREQUENT?")
    crime_counts = data["primary_type"].value_counts()
    most_common = crime_counts.index[0]
    print(f"Most frequent crime category: {most_common}")
```

Output:

```
Most frequent crime category: THEFT
```

Insight:

- The most frequent crime category is identified using the highest count of primary_type.
- This shows which type of crime contributes the largest number of records in the dataset.

Q2. Is the Arrest Rate Consistent Across Different Years?

Code :

```
print_task("Q2.", "IS THE ARREST RATE CONSISTENT ACROSS DIFFERENT YEARS?")
yearly_arrests = (data.groupby("Year")["arrest"]
                  .mean().mul(100).round(2))
print(yearly_arrests.to_string())
difference = (yearly_arrests.max() - yearly_arrests.min())
if difference <= 5:
    print("Arrest rate is relatively consistent.")
else:
    print("Arrest rate is not consistent.")
```

Output:

```
Year
2015    27.75
2016    25.62
2017    29.17
2018    33.80
2019    31.78
2020    30.36
2021    29.69
2022    32.58
2023    27.78
Arrest rate is not consistent.
```

Insight:

- The arrest percentage is calculated separately for each year.
- The difference between the highest and lowest yearly arrest rates is used to determine consistency.
- A smaller difference indicates a relatively consistent arrest rate, while a larger difference indicates greater variation.

Q3. Which Month Has the Highest Crime Frequency?

Code :

```
print_task("Q3.", "WHICH MONTH HAS THE HIGHEST CRIME FREQUENCY?")
month_frequency = (data["Month"].value_counts().sort_index())
highest_month = month_frequency.idxmax()
print(f"Month: {highest_month}")
```

Output:

```
| Month: 10
```

Insight:

- Crime records are grouped according to the month.
- The month with the largest number of records is identified as the month with the highest crime frequency.
- This helps identify the period with the greatest reported crime activity.

Conclusion :

- Use Case 2 analyzes Chicago crime patterns from multiple perspectives, including yearly trends, crime categories, arrests, time-based patterns, and community areas.
- The visualizations make it easier to identify important crime patterns in the dataset.
- The analysis also provides answers to specific questions about the most frequent crime, arrest consistency, and highest-crime month.